
Formalizing Brouwer’s Fixed Point Theorem via Scarf’s Lemma in Lean

Anonymous Authors¹

Abstract

We formalize in Lean 4 a combinatorial route from Scarf’s theorem to Brouwer’s fixed point theorem and the existence of mixed Nash equilibria in finite games. The development follows Ivanov’s indexed-order formulation of Scarf’s theorem, derives Brouwer’s theorem for the standard simplex through finite grids, compactness, continuity, and vanishing-diameter estimates, and extends the result to finite products of simplices by an explicit embedding–projection construction. We then apply the product theorem to mixed strategy spaces of finite games via a continuous Nash map whose fixed points are Nash equilibria. In addition, we introduce Brouwer-Bench, an 80-item Lean-grounded benchmark built from the formalization to evaluate whether language models can explain the roles of definitions, lemmas, and proof endpoints in the Scarf–Brouwer–Nash pipeline. The project thus contributes both a machine-checked formalization and a small evaluation suite for proof-structure understanding in Lean developments.

1. Introduction

Brouwer’s fixed point theorem originated as a fundamental result in topology (Brouwer, 1911) and has since become a central existence tool in analysis, economics, and game theory (Border, 1985). One of its most important applications is the existence of mixed Nash equilibria in finite games (Nash, 2024): the mixed strategy space of a finite game is a finite product of simplices, and a suitably constructed continuous self-map of this space has a fixed point precisely when the corresponding mixed strategy profile is an equilibrium. Classical proofs of Brouwer’s theorem often rely on topological tools such as homology, degree theory, or simplicial approximation. For a formal proof,

however, it is useful to expose a route whose finite combinatorial core can be separated from the analytic limiting argument and then reused in the game-theoretic endpoint.

This paper follows such a route through Scarf’s combinatorial theorem (Scarf, 1982). In Ivanov’s abstract formulation (Ivanov, 2019), Scarf’s theorem concerns a finite set equipped with indexed linear orders and a coloring. The proof is organized around dominant sets, cells, rooms, and doors. The key structural fact is a room–door incidence property: outside doors are incident to exactly one room, while internal doors are incident to exactly two rooms. This incidence structure supports a parity argument, which yields the existence of a colorful room. Applying Scarf’s theorem to increasingly fine grids of the standard simplex gives the finite combinatorial input for Brouwer’s fixed point theorem. The passage from colorful rooms to a fixed point is then obtained by compactness, continuity, and vanishing-diameter estimates. Finally, Brouwer’s theorem for the standard simplex is extended to finite products of simplices by an explicit embedding–projection construction, giving the fixed-point principle needed for the game-theoretic application.

The paper has two main contributions. First, it gives a Lean 4 formalization of the full Scarf–Brouwer–Nash pipeline, including the combinatorial parity argument, the analytic limiting step, the product-of-simplices reduction, and the Nash-map endpoint. Second, it turns the resulting proof development into BrouwerBench, a small benchmark for Lean-grounded proof-structure understanding. Rather than asking models to generate complete Lean proofs, Brouwer-Bench asks them to explain the roles of local definitions, lemmas, and proof endpoints using the formal context supplied by the development. This provides an intermediate evaluation target for auto-formalization workflows, where models often need to read existing Lean code and reason about proof architecture before producing kernel-checkable terms.

1.1. Lean and Mathlib

Throughout the paper, we use Lean 4 (Moura & Ullrich, 2021) as the proof assistant in which all definitions, theorem statements, and proofs are checked, and Mathlib (The mathlib Community, 2020) as the underlying mathematical

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

library.

Our development builds on Mathlib’s infrastructure for finite types and finite sets, real numbers, continuity, compactness, finite-dimensional spaces, and the standard simplex, while introducing the Scarf-specific objects needed for the formalization, including indexed linear orders, dominant sets, cells, rooms, doors, colorful rooms, and the room–door incidence relation.

Throughout the text, we use the clickable symbol  to link mathematical statements to their corresponding Lean source code.

1.2. Paper Outline

In §2, we situate the work relative to Scarf-style combinatorial fixed-point arguments, formalized mathematics in Lean, and formal-mathematics benchmarks. In §3, we describe the formalization of Scarf’s combinatorial theorem: after introducing the indexed-order setting and the basic notions of dominant sets, cells, rooms, and doors, we prove the room–door incidence properties and use them to develop the coloring and parity argument for colorful rooms. §4 is devoted to the derivation of Brouwer’s fixed point theorem from Scarf’s theorem: in §4.1, we treat the standard simplex by finite grid approximation, compactness, and continuity, and in §4.2, we extend the result to finite products of simplices through an explicit embedding–projection construction. In §5, we apply the product version of Brouwer’s theorem to the mixed strategy spaces of finite games and derive the existence of mixed Nash equilibria. In §6, we introduce BrouwerBench and report model results. Finally, we conclude with implementation remarks, limitations, and future work in §7.

2. Related Work

Combinatorial fixed-point arguments. Brouwer’s fixed point theorem is a classical cornerstone of topology (Brouwer, 1911) and a standard tool in economics and game theory (Border, 1985). The equilibrium application goes back to Nash’s proof of existence for finite games (Nash, 2024), where mixed strategy profiles live in products of simplices. Our formalization follows the combinatorial tradition represented by Scarf’s theorem (Scarf, 1982): instead of proving Brouwer by homological or degree-theoretic methods, it derives the fixed-point theorem from finite cells, rooms, doors, and a parity argument. Ivanov’s indexed-order formulation (Ivanov, 2019) is especially well suited to formalization because it isolates the finite combinatorial structure before the analytic limiting step.

Game theory and fixed points. Fixed-point methods are central in mathematical game theory, and product-of-simplices arguments are the standard bridge from topology to mixed Nash equilibria. Textbook treatments develop finite games, mixed strategies, expected payoffs, and equilibrium existence as part of a broader mathematical theory (Laraki et al., 2019). Our contribution is narrower but more formal: we build the specific fixed-point pipeline needed for finite games inside Lean, including the product-of-simplices reduction and the Nash map whose fixed points are equilibria.

Lean formalization. The development is written in Lean 4 (Moura & Ullrich, 2021) and builds on Mathlib (The mathlib Community, 2020). Mathlib provides the surrounding infrastructure for finite types, finite sets, real analysis, compactness, continuity, finite-dimensional spaces, and standard simplices. The new material in this work lies in assembling these components into a full proof chain: Scarf’s finite incidence argument, the grid-to-limit proof of Brouwer, the explicit product embedding and projection, and the mixed-game endpoint.

Formal-mathematics benchmarks and auto-formalization. Recent work on neural theorem proving and formal-mathematics benchmarks has emphasized tasks such as proof search, theorem proving, and olympiad-style formal problem solving (Polu & Sutskever, 2020; Zheng et al., 2021). BrouwerBench is complementary to these end-to-end evaluations. It does not ask a model to produce a complete Lean proof. Instead, it evaluates whether a model can read local Lean-grounded context and explain how definitions and lemmas function inside an existing proof development. This targets a practical intermediate capability for auto-formalization: understanding and navigating a formal library before attempting proof synthesis or proof repair.

3. Formalizing Scarf’s Combinatorial Theorem

This section contains the finite combinatorial part of the development. We follow Ivanov’s abstract formulation of Scarf’s theorem (Ivanov, 2019), where the input is a finite set equipped with a family of indexed linear orders and a coloring. The proof is organized around three layers. First, dominance turns order-theoretic data into finite cells. Second, rooms and doors provide the incidence structure used for counting. Third, colorings and nearly colorful cells turn this incidence structure into a parity argument producing a colorful room.

3.1. Orders, Dominant Sets, and Cells

We work with a finite type T together with a finite index type I . For each $i \in I$, we are given a linear order $<_i$ on T . The indexed family of orders is the basic object from which dominance is defined. In Lean, this data is bundled as a typeclass [\[1\]](#), so that the notation for comparisons can depend on the index.

Definition 3.1 (Indexed family of linear orders). A family of linear orders on T indexed by I is a collection $(<_i)_{i \in I}$ such that each $<_i$ is a linear order on T .

```
class IndexedLOrder (I T : Type*) where
  IST : I → LinearOrder T
```

Listing 1. Indexed families of linear orders.

Given a nonempty subset $\sigma \subseteq T$ and an index $i \in I$, we write $\min_i \sigma$ for the minimum of σ with respect to $<_i$. Dominance says that every point of T is controlled by at least one of the orders indexed by C . This is the order-theoretic condition that later forces the relevant finite sets to have the correct cardinalities.

Definition 3.2 (Dominant set). Let C be a nonempty subset of I . A subset σ of T is dominant with respect to C if, for every element y of T , one can find an index i in C such that y is below every element of σ in the order $<_i$.

When σ is nonempty, this is equivalent to saying that no element of T is strictly larger than all the minima $\min_i \sigma$ for i in C .

```
def isDominant := ∀ y, ∃ i ∈ C, ∀ x ∈ σ,
  y ≤ [i] x
```

Listing 2. Dominance.

We adopt the convention that $\emptyset \subset T$ is dominant with respect to every non-empty subset $C \subset I$ [\[2\]](#). The following lemma collects the basic closure and cardinality facts about dominant sets. These facts are used repeatedly when passing between cells, rooms, and doors.

Lemma 3.3 (Basic properties of dominant sets). If σ is nonempty and dominant with respect to C , then $\sigma = \{\min_i \sigma \mid i \in C\}$, and hence $|\sigma| \leq |C|$ [\[3\]](#) and hence $|\sigma| \leq |C|$ [\[4\]](#). If $\tau \subset \sigma$, then τ is dominant with respect to C [\[5\]](#), and if $C \subset D$, then σ is dominant with respect to D [\[6\]](#).

The preceding lemma allows us to treat dominant pairs (σ, C) as cells of a finite combinatorial object. Rooms are the full-dimensional cells, while doors are their codimension-one boundaries in the counting argument.

Definition 3.4 (Cells, rooms, and doors). A *cell* is a pair (σ, C) where σ is dominant with respect to C . A cell is called a *room* if $|C| = |\sigma|$, and a *door* if $|C| = |\sigma| + 1$.

```
abbrev isCell := isDominant σ C
abbrev isRoom := isCell σ C ∧ C.card = σ.card
abbrev isDoor := isCell σ C ∧ C.card
  = σ.card + 1
```

Listing 3. Cells, rooms, and doors.

3.2. Rooms, Doors, and the Two-Room Property

The next step is to make precise how rooms meet along doors. In the formalization, this is captured by the predicate `isDoorof` [\[7\]](#), which records that a door is obtained from a room either by removing an element from the underlying set or by adding a color. This relation is the local incidence relation used later in the parity count.

Definition 3.5 (Outside and internal doors). A door (τ, D) is called an *outside door* if $\tau = \emptyset$, and an *internal door* otherwise.

```
variable (τ D) in
abbrev isOutsideDoor := IST.isDoor τ D ∧ τ
  = Finset.empty

variable (τ D) in
abbrev isInternalDoor := IST.isDoor τ D
  ∧ τ.Nonempty
```

Listing 4. outside door and internal door.

The following two lemmas give the degree structure of this incidence relation. Outside doors behave as boundary objects: they have degree one. Internal doors behave as shared faces: they have degree two.

Lemma 3.6 (Outside-door property). Suppose that $|D| = 1$. Then $(\emptyset, D)$ is an outside door [\[8\]](#) and is a door of exactly one room. Every outside door has this form [\[9\]](#).

Lemma 3.7 (Two-room property). Every internal door belongs to exactly two rooms [\[10\]](#).

Thus the rooms and doors form a finite graph-like incidence structure: starting from an outside door, one enters a unique room, and every time one crosses an internal door there is a unique other room on the opposite side. This is the combinatorial shape behind the later path-following and parity argument.

The proof of the two-room property is the most involved local part of the Scarf formalization. It analyzes an internal door (τ, D) via two complementary constructions: either removing a color from D [\[11\]](#), or extending τ by a suitable element [\[12\]](#). These are implemented in Lean via the auxiliary constructions `M_set` [\[13\]](#) and maximal elements `m_element` [\[14\]](#), which ensure that both constructions yield valid rooms. In the paper, we use the two-room property as

a black-box incidence theorem; in Lean, the proof supplies the exact finite-set witnesses needed to make the incidence count well typed.

3.3. Colorings and the Parity Argument

We now add the coloring data. The coloring selects, for each element of T , an index in I . A colorful cell is one whose index set is exactly the image of its underlying set under this coloring. Nearly colorful cells differ from being colorful by exactly one missing color; the missing color is recorded as the type. This typed version is useful because the parity argument counts objects of a fixed type.

Definition 3.8 (Coloring). Fix a coloring $c : T \rightarrow I$. A cell (σ, C) is called *colorful* if $C = c(\sigma)$, and *nearly colorful* if $|C \setminus c(\sigma)| = 1$. For a nearly colorful cell (σ, C) , the unique element of $C \setminus c(\sigma)$ is called its *type*.

```
def isColorful : Prop := IST.isCell σ C
  ∧ σ.image c = C

def isNearlyColorful : Prop := IST.isCell σ C
  ∧ (C \ σ.image c).card = 1

def isTypedNC (i : I) (σ : Finset T)
  (C : Finset I) : Prop := IST.isCell σ C
  ∧ (C \ (σ.image c)) = {i}
```

Listing 5. Colorful and nearly colorful cells.

The next lemmas connect the uncolored incidence structure with the coloring data. They identify the boundary terms, describe how colorful rooms contribute nearly colorful doors, and show that nearly colorful rooms have the expected number of nearly colorful doors.

Lemma 3.9 (Outside doors and doors of colorful rooms). *For every $i \in I$ there is exactly one outside door of the type i . Every door of a colorful room is nearly colorful and there is exactly one door of each type among them.*

Lemma 3.10 (Nearly colorful door transition). *Suppose that (τ, D) is a nearly colorful door of a room (σ, C) . Then (σ, C) is either colorful, or nearly colorful of the same type as (τ, D) .*

Lemma 3.11 (Nearly colorful cardinality properties). *If (σ, C) is a nearly colorful room, then $|c(\sigma)|$ is equal either to $|\sigma|$ or to $|\sigma| - 1$. If (σ, C) is a nearly colorful door, then $c(\sigma) \subset C$.*

Lemma 3.12 (Doors of nearly colorful rooms). *If (σ, C) is a nearly colorful room, then (σ, C) has two nearly colorful doors.*

Together, these lemmas turn the room–door incidence relation into a parity count. Fixing a type i , the unique outside door of type i contributes one boundary term. Internal

nearly colorful doors are counted twice because of the two-room property. Nearly colorful rooms also contribute an even number of typed doors. Therefore the remaining contribution comes from colorful rooms, forcing the relevant colorful-room count to be odd.

Theorem 3.13 (Scarf’s combinatorial theorem). *For every coloring $c : T \rightarrow I$, there exists a colorful room. Moreover, the number of colorful rooms is odd.*

This theorem completes the finite combinatorial core of the formalization. In the next section, the finite set T will be instantiated by a grid in the standard simplex. The colorful rooms supplied by Scarf’s theorem then become the discrete approximations from which Brouwer fixed points are obtained by a limiting argument.

4. From Scarf’s Theorem to Brouwer’s Fixed Point Theorem

This section derives Brouwer’s fixed point theorem from the formalized Scarf theorem. The argument has two stages. We first prove Brouwer’s theorem for the standard simplex by applying Scarf’s theorem to increasingly fine finite grids and then passing to a limit. We then obtain the corresponding theorem for finite products of simplices by reducing the product case to the standard-simplex case through an explicit embedding–projection construction.

4.1. Brouwer’s Theorem on the Standard Simplex

Fix a positive integer n , and let Δ^{n-1} be the standard simplex with coordinates indexed by $\text{Fin } n$. The goal is to prove that every continuous self-map

$$f : \Delta^{n-1} \rightarrow \Delta^{n-1}$$

has a fixed point. The proof does not use Scarf’s theorem directly on the simplex itself. Instead, for each positive integer l , we work with a finite grid T_l of rational points whose coordinates have denominator l . Informally,

$$T_l = \left\{ x \in \Delta^{n-1} \mid x_i \in \frac{1}{l}\mathbb{Z} \text{ for all } i \right\}.$$

In Lean, the grid is represented by integer coordinates in $\text{Fin } (1+1)$ whose sum is l ; the corresponding point of the simplex is obtained by dividing each coordinate by l . The subtype TT defines the finite grid, while TTtoStdSimplex and the coercion instance TT.CoestdSimplex realize a grid point as an element of the standard simplex.

```
abbrev TT := {x : Π₁ ( _ : Fin n), Fin (1+1) | ∑
  i, (x i : ℕ) = 1}

def TTtoStdSimplex (x : TT n 1) : stdSimplex ℝ
  (Fin n)
```

Listing 6. The discrete grid used to approximate the simplex.

4.1.1. THE ORDERED GRID AND DOMINANCE ESTIMATES

To apply Scarf's theorem, each grid T_l is equipped with an indexed family of linear orders, one order for each coordinate. For a fixed coordinate i , the order first compares the i -th coordinate and then uses a lexicographic tie-breaker on the whole grid point. This is encoded by `TT.ILt` and assembled into the indexed linear order `TT.ILO` [\[1\]](#). These orders are chosen so that dominance in the sense of Scarf's theorem imposes strong metric restrictions on the corresponding grid points. This is the point at which the abstract combinatorial theorem becomes a quantitative statement about the simplex.

```
abbrev TT.ILt (x y : TT n 1) :=
  toLex (x i, x) < toLex (y i, y)

instance TT.ILO : IndexedLOrder (Fin n) (TT n 1)
```

Listing 7. The indexed linear orders on the grid.

The key quantitative input is that dominance on the grid gives metric control. A dominant set has small diameter, and the coordinates outside its color set are uniformly small. These estimates are the bridge between the finite combinatorial theorem and the limiting argument.

Theorem 4.1 (Dominance estimates on the grid). *Let $\sigma \subseteq T_l$ and $C \subseteq \text{Fin } n$. If σ is dominant with respect to C , then*

$$|x_i - y_i| < \frac{2(n+1)}{l}$$

for every $x, y \in \sigma$ and every $i \in \text{Fin } n$ [\[1\]](#). Moreover, if $i \notin C$, then

$$x_i < \frac{n+1}{l}$$

for every $x \in \sigma$ [\[1\]](#).

Thus, as l tends to infinity, dominant rooms become small, and the coordinates not belonging to their color sets vanish in the limit.

4.1.2. THE COLORING INDUCED BY A SELF-MAP

Let $f : \Delta^{n-1} \rightarrow \Delta^{n-1}$ be continuous. For any point $x \in \Delta^{n-1}$, the coordinate sums of x and $f(x)$ are both equal to 1. Hence not all coordinates can strictly decrease: there exists an index i such that

$$x_i \leq f(x)_i.$$

The formalization packages this elementary observation as the nonemptiness lemma `stdSimplex.upidx` and the choice function `stdSimplex.pick` on the simplex. Restricting this choice to the grid T_l gives the coloring required by Scarf's theorem, namely `Fcolor`.

```
noncomputable def stdSimplex.pick
  (x y : stdSimplex ℝ (Fin n))

def Fcolor (x : TT n 1) : Fin n :=
  stdSimplex.pick x (f x)
```

Listing 8. The coloring induced by a continuous self-map.

Applying the formalized Scarf theorem to `Fcolor` produces a colorful room on each grid T_l . In Lean, these rooms are chosen as a sequence `room_seq`, and a representative point from each room is chosen as `room_point_seq`. This sequence of rooms is the bridge between the finite combinatorial theorem and the limiting proof of Brouwer's theorem.

```
def room_seq (l' : ℕ) :=
  let l : PNat := ⟨l'+1, Nat.zero_lt_succ _⟩
  Classical.choice (TT.ILO.Scarf (@Fcolor n 1 f)).to_subtype

def room_point_seq (l' : ℕ) :=
  pick_colorful_point
  (Finset.mem_filter.1 (room_seq f l')).2.2 |>.1
```

Listing 9. Choosing colorful rooms and representative points.

4.1.3. COMPACTNESS AND THE LIMITING ARGUMENT

Choose one representative point from the colorful room obtained at each grid level. Since there are only finitely many possible color sets, we first pass to a subsequence on which the color set of the chosen colorful rooms is constant. This finite-image subsequence extraction is formalized by `exists_subseq_constant_of_finite_image` [\[1\]](#); the resulting constant color set and order embedding are packaged as `gpkg` and `g1` [\[1\]](#). We then use compactness of the standard simplex to extract a convergent subsequence of the corresponding room points; this step is packaged in Lean as `hpkg_aux` and `hpkg` [\[1\]](#). Let $z \in \Delta^{n-1}$ denote the resulting limit, and write the constant color set as C .

The dominance estimates now control the limit. First, the diameters of the selected colorful rooms tend to zero [\[1\]](#). Therefore every point in the same selected room converges to the same limit z . Second, for each coordinate $i \notin C$, the outside-coordinate estimate gives convergence to zero, and hence

$$z_i = 0 \quad \text{for all } i \notin C \text{ [\[1\]](#) .}$$

Since z lies in the simplex, it follows that

$$\sum_{i \in C} z_i = 1.$$

On the other hand, the coloring rule says that for each color $i \in C$, the chosen room contains grid points whose i -th

coordinate does not decrease under f . Passing to the limit and using continuity of f gives

$$f(z)_i \geq z_i \quad \text{for all } i \in C \text{ } \square.$$

Finally, $f(z)$ is also a point of the simplex. The inequalities on C , together with $\sum_{i \in C} z_i = 1$, force equality on all coordinates in C and force all coordinates outside C to be zero. Hence $f(z) = z$.

Theorem 4.2 (Brouwer’s fixed point theorem for the standard simplex). *Every continuous map $f : \Delta^{n-1} \rightarrow \Delta^{n-1}$ has a fixed point* $\square$.

4.2. Brouwer’s Theorem on Products of Simplices

We next extend Brouwer’s fixed point theorem from the standard simplex to finite products of simplices. This is the form needed for the game-theoretic application, where a mixed strategy profile is a point in a product of simplices. The formal proof reduces the product case to the standard-simplex case by constructing an explicit embedding–projection pair between the product and a larger simplex.

4.2.1. REDUCING PRODUCTS TO A STANDARD SIMPLEX

Let I be a finite index type, and let $\text{card}(i)$ be a positive integer for each $i \in I$. The product space considered in Lean is

$$\prod_{i \in I} \Delta^{\text{card}(i)-1}.$$

It is represented by the type `ProductSimplices card`. The ambient simplex has total coordinate number

$$N = \sum_{i \in I} \text{card}(i),$$

and is represented by `BigSimplex card` $\square$.

The main bookkeeping issue is that coordinates in the ambient simplex are indexed by a single flat index, while coordinates in the product are indexed by a pair consisting of a component $i \in I$ and a coordinate inside the i -th simplex. In Lean, this conversion is handled by `index_split` and `index_combine` $\square$. These maps allow the formalization to move between the block decomposition of the product and the flat coordinate system of the larger simplex.

4.2.2. THE EMBEDDING-PROJECTION CONSTRUCTION

The embedding sends a point of the product of simplices to a point of the ambient simplex by placing each component simplex into its corresponding coordinate block. The i -th block is rescaled by the weight $\text{card}(i)/N$, so that the total sum of all coordinates is one. This construction is formalized as `embed_from_product` $\square$.

The projection in the opposite direction is more delicate. Starting from a point of the ambient simplex, the natural idea is to recover each component simplex by normalizing the corresponding coordinate block. However, this normalization is not well-defined if one of the blocks has total mass zero. To avoid this degenerate case, the formal construction first pushes the ambient point toward a fixed uniform point `z_uniform`. This auxiliary operation is implemented by `pushTowardsZ`, together with the definitions `deficit` and `tPush` $\square$. The key property is that after this push, every coordinate block has positive total mass $\square$. The projection is then obtained by normalizing each block, and is formalized as `project_to_product` $\square$.

The central property of the construction is the retraction identity

$$\text{project_to_product} \circ \text{embed_from_product} = \text{id}.$$

For points already coming from the product, each coordinate block has the prescribed positive total mass, so the projection recovers the original product point. This identity is formalized by `project_embed_id` $\square$.

4.2.3. THE PRODUCT FIXED-POINT THEOREM

It remains to transfer Brouwer’s theorem from the ambient simplex back to the product. The embedding and projection are proved continuous as `embed_continuous` and `project_continuous` $\square$. Given a continuous map

$$f : \prod_{i \in I} \Delta^{\text{card}(i)-1} \rightarrow \prod_{i \in I} \Delta^{\text{card}(i)-1},$$

we form a continuous self-map of the ambient simplex by composing projection, f , and embedding:

$$F = \text{embed_from_product} \circ f \circ \text{project_to_product}.$$

The standard-simplex version of Brouwer’s theorem gives a fixed point of F . Projecting this fixed point back to the product and using the retraction identity gives a fixed point of f .

Theorem 4.3 (Brouwer fixed point theorem for products of simplices). *Every continuous self-map of a finite product of simplices has a fixed point* $\square$.

5. Nash Equilibrium Existence in Finite Games

We now apply the product version of Brouwer’s fixed point theorem to finite strategic-form games. The main point is that, after passing from pure strategies to mixed strategies, the strategy space becomes a finite product of simplices. We then construct the standard Nash map on this product, prove that it is continuous, and show that every fixed point of this map satisfies the mixed Nash equilibrium condition.

5.1. Finite Games and Mixed Strategy Profiles

A strategic-form game consists of a nonempty type of players, a nonempty type of pure strategies for each player, and a payoff function for each player. In Lean, the basic structure is kept deliberately general: finiteness is added only in the derived structure `FinGame`. This separation allows the same notion of game to be reused before and after passing to mixed strategies.

```
structure Game where
  I : Type*
  HI : Inhabited I
  SS : I → Type*
  HSS (i : I) : Inhabited (SS i)
  g : I → (Π i, SS i) → ℝ

attribute [instance] Game.HI Game.HSS
```

Listing 10. Strategic-form games

For a game $G = (I, (S^i)_{i \in I}, (g^i)_{i \in I})$, a pure Nash equilibrium is a strategy profile $s = (s^i)_{i \in I}$ such that no player can improve their payoff by changing only their own strategy: for every player i and every alternative pure strategy $t^i \in S^i$,

$$g^i(t^i, s^{-i}) \leq g^i(s^i, s^{-i}).$$

This condition is formalized as the basic equilibrium predicate for games.

For the fixed-point argument we restrict to finite games. A finite game has finitely many players and finitely many pure strategies for each player. Its mixed strategy space is the product of standard simplices

$$\Delta(G) = \prod_{i \in I} \Delta(S^i),$$

where a point $\sigma \in \Delta(G)$ assigns to every player i a probability distribution σ^i over S^i . In Lean this is represented by the dependent function type `mixedS`.

```
structure FinGame extends Game where
  FinI : Fintype I
  FinSS : ∀ i : I, Fintype (SS i)

attribute [instance] FinGame.FinI FinGame.FinSS

variable {G : FinGame}

instance {G : FinGame} : Fintype G.I := G.FinI
instance {G : FinGame} {i : G.I} : Fintype (G.SS i) := G.FinSS i

variable (G) in
abbrev mixedS := (i : G.I) → stdSimplex ℝ (G.SS i)
```

Listing 11. Finite games and mixed strategy profiles

The pure payoff function extends to mixed profiles by taking expected payoff. For a player i , the mixed payoff `mixed_g i σ` is the multilinear extension of the pure payoff function, obtained by summing over pure strategy profiles with weights given by the players' mixed strategies. The formalization also proves the algebraic lemmas needed to treat this extension as linear in the relevant coordinates.

It is convenient to package the mixed extension itself as another game, whose strategy type for player i is the simplex over S^i and whose payoff function is the expected payoff.

```
def FinGame2MixedGame (G : FinGame) : Game := {
  I := G.I
  HI := G.HI
  SS := fun i => S (G.SS i)
  HSS := inferInstance
  g := mixed_g
}
```

Listing 12. The mixed extension of a finite game

A mixed Nash equilibrium is then a Nash equilibrium of this mixed game: a mixed profile σ^* such that no player can obtain a larger expected payoff by replacing σ^{*i} with another mixed strategy while all other players keep their components fixed.

5.2. The Nash Map

Following the standard Brouwer proof of Nash's theorem, we define a continuous self-map on the mixed strategy space. For a mixed profile σ , player i , and pure strategy $s^i \in S^i$, let

$$A_i(s^i, \sigma) = (g^i(s^i, \sigma^{-i}) - g^i(\sigma))^{+}, \quad a^{+} = \max(a, 0).$$

The Nash map $F : \Delta(G) \rightarrow \Delta(G)$ is defined coordinate-wise by

$$F(\sigma)^i(s^i) = \frac{\sigma^i(s^i) + A_i(s^i, \sigma)}{1 + \sum_{t^i \in S^i} A_i(t^i, \sigma)}.$$

The denominator is positive, and the coordinates in each player block sum to one; hence $F(\sigma)^i$ is again a mixed strategy. The Lean definition mirrors this formula by first defining the unnormalized numerator and then normalizing it inside the simplex certificate.

```
noncomputable abbrev g_function (i : G.I) (σ :
  G.mixedS) (a : G.SS i) : ℝ :=
  σ i a + max 0 (mixed_g i (Function.update σ i
    (stdSimplex.pure a)) - mixed_g i σ)

noncomputable abbrev nash_map_aux (σ :
  G.mixedS) (i : G.I) (a : G.SS i) : ℝ :=
  g_function i σ a / ∑ b : G.SS i, g_function
    i σ b
```

```

385 noncomputable def nash_map ( $\sigma$  : G.mixedS) :
386   G.mixedS :=
387   fun i  $\mapsto$  (nash_map_aux  $\sigma$  i, nash_map_cert  $\sigma$  i)

```

Listing 13. The Nash map

The proof has two parts. First, we prove that `nash_map` is continuous. Mathematically this follows from the finiteness of the strategy sets and the fact that the formula is built from finite sums, subtraction, max, and division by a positive denominator. In Lean, the main work is to make the continuity of the expected payoff function explicit before assembling the continuity proof for the whole map. [↗](#)

Second, applying the product version of Brouwer’s theorem to `nash_map` gives a fixed point σ . [↗](#) It remains to show that this fixed point is a mixed Nash equilibrium. If all positive payoff improvements vanish, then every pure deviation has payoff at most $g^i(\sigma)$, and therefore no mixed deviation can improve the payoff. If some positive improvement were available at a fixed point, the normalization equation defining the Nash map would force a contradiction with the expected payoff identity. Thus every fixed point of the Nash map satisfies the mixed Nash equilibrium condition.

Theorem 5.1 (Existence of mixed Nash equilibria). *Every finite strategic-form game has a mixed Nash equilibrium.* [↗](#)

6. BrouwerBench: A Lean-Grounded Benchmark

The same formalization also provides a compact benchmark for evaluating how well language models understand Lean-grounded mathematical developments. We call this benchmark *BrouwerBench*. Its purpose is not to measure end-to-end proof generation: the models are not asked to produce Lean terms that are compiled by the kernel. Instead, the benchmark measures whether a model can read local Lean context, identify the role of definitions and lemmas, and explain how proof components fit together across the Scarf–Brouwer–Nash pipeline.

6.1. Benchmark Construction

BrouwerBench v1 contains 80 question–answer items extracted from the formalization. The items cover four parts of the development: Scarf’s theorem, the standard-simplex proof of Brouwer’s theorem, the product-of-simplices extension, and the Nash equilibrium application. Each item contains a short task-specific Lean-grounded context, a natural-language question, a reference answer, evidence pointers to the Lean source, and a 0–2 grading rubric. The questions are organized by proof role, including definition roles, construction roles, proof dependencies, parity arguments, analytic limiting steps, theorem summaries, and

endpoint connections.

During evaluation, each model receives the section-level Lean context together with the task-specific context and question. The section-level context includes the relevant definitions, structures, and abbreviations, such as the Scarf cell/room/door predicates, the finite-grid constructions used in Brouwer’s theorem, the embedding–projection maps for products of simplices, and the Nash map definitions. The answer is then scored manually according to the item rubric: 0 means that the answer misses the main mathematical point, 1 means that it identifies part of the intended structure but omits an essential dependency, and 2 means that it explains the relevant Lean-grounded proof role accurately enough to match the reference answer.

6.2. Models and Results

We evaluated four locally deployed models: qwen3:8b as a small baseline, gemma3:12b, kimina-prover:7b, and gpt-oss:20b. The results in Table 1 use the 80-item BrouwerBench v1 dataset, with a maximum score of 160 points.

The strongest model, gpt-oss:20b, scores 76.2% overall. It performs especially well on theorem-summary and proof-dependency questions, where the task is to connect named Lean lemmas to the surrounding proof strategy. The smaller qwen3:8b baseline remains competitive at 63.1%, especially on product and Nash questions, suggesting that this benchmark is not simply measuring model size. gemma3:12b is a useful newer general control but trails the baseline on definition-heavy Scarf and Nash dependencies. kimina-prover:7b, despite being prover-oriented, performs substantially worse on this benchmark, which is consistent with the benchmark’s emphasis on explanatory proof-structure understanding rather than low-level proof search.

6.3. Interpretation

These results should be interpreted as evidence about Lean-grounded proof understanding, not as direct evidence about the ability to synthesize complete formal proofs. A high-scoring answer typically names the relevant Lean objects and explains why they are needed: for example, why `typed_colorful_room_odd c default` uses an inhabited default type, why Brouwer’s limiting argument first stabilizes a color/index set, why `project_embed_id` is the retraction identity needed in the product theorem, or how `mixed_g_linear` moves the Nash proof from pure deviations to mixed deviations. These are intermediate reasoning skills that support auto-formalization workflows, but they do not by themselves certify a Lean proof.

The benchmark is therefore complementary to end-to-

Table 1. BrouwerBench v1 results. Scores use a 0–2 rubric per item.

Model	Score	Accuracy	Runtime (s)	Avg. / task (s)
gpt-oss:20b	122/160	76.2%	1448.1	18.10
qwen3:8b	101/160	63.1%	428.1	5.35
gemma3:12b	89/160	55.6%	822.8	10.29
kimina-prover:7b	58/160	36.2%	986.4	12.33

Table 2. BrouwerBench v1 scores by formalization section.

Model	Scarf	Brouwer	Product Brouwer	Nash
gpt-oss:20b	40/48	25/40	29/36	28/36
qwen3:8b	30/48	22/40	25/36	24/36
gemma3:12b	25/48	21/40	24/36	19/36
kimina-prover:7b	14/48	12/40	14/36	18/36

end theorem-proving evaluations. End-to-end tasks test whether a model can produce code accepted by Lean; BrouwerBench tests whether a model can explain the architecture of an existing formal proof from local formal context. For a formalization project like ours, this distinction is useful: many practical proof-assistant workflows require models to read existing definitions, identify reusable lemmas, explain failures, and suggest proof plans before a final proof script is produced.

7. Discussion

7.1. Implementation Remarks

The formalization is organized as a chain of reusable components, moving from finite combinatorics to fixed-point theory and then to equilibrium existence. The Scarf part is the most combinatorial. Although the informal proof is guided by an intuitive room–door picture, the Lean development requires explicit definitions of indexed orders, dominant sets, cells, rooms, doors, and the incidence relation between them. Many steps that are implicit on paper become finite-set and cardinality arguments in Lean. In particular, the two-room property requires a careful case analysis of outside and internal doors, together with explicit witnesses for the possible rooms adjacent to a given door.

The Brouwer part illustrates a different kind of formalization difficulty. The proof must connect the finite output of Scarf’s theorem with an analytic limiting argument. This requires defining finite grid approximations of the simplex, constructing the coloring induced by a continuous self-map, proving quantitative diameter estimates, extracting convergent subsequences, and transporting coordinate inequalities to the limit. The product-of-simplices version adds further bookkeeping: points in a product of simplices must be related to points in a larger ambient simplex, and the projection back to the product must avoid degenerate zero-block cases. Our embedding–projection construction

makes this reduction explicit and proves the required retraction identity inside Lean.

The Nash part then uses the product version of Brouwer’s theorem as a reusable fixed-point principle. Once mixed strategy profiles are represented as products of simplices, the remaining tasks are to define expected payoffs, construct the Nash map, prove its continuity, and show that its fixed points satisfy the mixed Nash equilibrium condition. This separation is useful: the game-theoretic proof can be developed mostly in terms of payoff functions and best-response inequalities, while the topological fixed-point argument is isolated in the preceding Brouwer formalization.

The benchmark follows the same modular structure. Because the proof is organized around named definitions, lemmas, and endpoints, each stage naturally yields questions about proof roles: what a definition packages, why a lemma is needed, and how a local proof component contributes to the global Scarf–Brouwer–Nash chain. This makes BrouwerBench a by-product of the formalization rather than a separate natural-language dataset. The benchmark also exposes a practical lesson for auto-formalization: understanding the architecture of an existing Lean development is a distinct skill from producing a final proof term, and both skills are needed in realistic proof-assistant workflows.

7.2. Limitations

The current benchmark is intentionally focused. BrouwerBench v1 contains 80 items from one formalization pipeline, so the results should not be read as a general ranking of theorem-proving systems or Lean models. The questions are also explanatory: they test whether a model can connect local Lean context to proof structure, not whether it can synthesize a complete proof accepted by Lean. This is why a prover-oriented model may score poorly here even if it is useful in a proof-search setting.

The scoring protocol is another limitation. Each item uses a 0–2 rubric and was manually graded. This makes the scores interpretable, but it also means that the benchmark depends on human judgment about partial credit. A larger evaluation should include multiple graders, adjudication for borderline cases, and a clearer separation between exact Lean-name recall and higher-level mathematical explanation. The present results are best viewed as a pilot study showing that the formalization can generate meaningful evaluation units.

7.3. Future Work

A natural direction for future work is to use this development as the starting point for a broader Lean library of game theory. The present formalization establishes several foundational components: finite games, mixed strategies as products of simplices, expected payoffs, Nash equilibria, and the fixed-point argument needed for equilibrium existence. These components could serve as a formal backbone for later developments covering zero-sum games, extensive-form games, repeated games, equilibrium refinements, and mechanism-design-style constructions.

Our longer-term goal is to formalize the material of a full textbook on game theory. A textbook-scale project would require not only more theorems, but also a more systematic hierarchy of reusable definitions, so that later chapters can build on earlier formalized notions without repeatedly rebuilding the same infrastructure. The Scarf–Brouwer–Nash pipeline provides a useful test case because it combines finite combinatorics, topology, analysis, and game theory in a single proof development.

Auto-formalization is an important tool for making such a project feasible. Recent work on neural theorem proving and formal-mathematics benchmarks has shown that language models can contribute to proof search, statement translation, and proof repair, while still relying on proof assistants as the final checkers (Polu & Sutskever, 2020; Zheng et al., 2021). In our setting, a possible workflow is to combine human-written mathematical outlines with model-generated Lean drafts: the system would translate textbook definitions and proof sketches into Lean, retrieve relevant Mathlib lemmas and previously formalized game-theoretic results, generate auxiliary lemmas when direct proof attempts fail, and iteratively repair proof scripts using Lean error messages.

BrouwerBench can be expanded alongside this formalization work. The next version should add more theorem families, separate proof-understanding questions from statement-translation and proof-synthesis tasks, and include end-to-end Lean tasks whose outputs are checked by the kernel. A useful long-term benchmark would therefore have several layers: natural-language-to-Lean state-

ment translation, lemma retrieval, proof-plan explanation, proof-script generation, and proof repair from Lean error messages. The current benchmark covers only the proof-understanding layer, but it gives a concrete starting point for evaluating models on the kind of local reasoning that precedes successful formal proof construction.

Impact Statement

This paper contributes to formalized mathematics and machine-checkable reasoning in proof assistants. Its results are primarily foundational and theoretical, focusing on the formal verification of a route from Scarf’s combinatorial theorem to Brouwer’s fixed point theorem and Nash equilibrium existence. We do not anticipate direct negative societal impacts from the results themselves. More broadly, the work may support the development of reliable mathematical libraries and auto-formalization tools; as with other formal reasoning technologies, their downstream use should be evaluated in the context of the systems in which they are deployed.

References

- Border, K. C. *Fixed point theorems with applications to economics and game theory*. Cambridge university press, 1985.
- Brouwer, L. E. Beweis der invarianz der dimensionenzahl. *Mathematische Annalen*, 70(2):161–165, 1911.
- Ivanov, N. V. Beyond Sperner’s lemma. *arXiv preprint arXiv:1902.00827*, 2019.
- Laraki, R., Renault, J., and Sorin, S. *Mathematical Foundations of Game Theory*. Universitext. Springer, Cham, 2019. ISBN 978-3-030-26645-5. doi: 10.1007/978-3-030-26646-2.
- Moura, L. d. and Ullrich, S. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, pp. 625–635. Springer, 2021.
- Nash, J. F. Non-cooperative games. In *The Foundations of Price Theory Vol 4*, pp. 329–340. Routledge, 2024.
- Polu, S. and Sutskever, I. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.
- Scarf, H. E. The computation of equilibrium prices: an exposition. *Handbook of mathematical economics*, 2: 1007–1061, 1982.
- The mathlib Community. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*,

CPP 2020, New Orleans, LA, USA, January 2020. ACM.
 doi: 10.1145/3372885.3373824. URL <https://doi.org/10.1145/3372885.3373824>.

Zheng, K., Han, J. M., and Polu, S. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*, 2021.

A. Formalization Overview

The Lean development is organized as a proof pipeline. The Scarf file establishes the finite combinatorial theorem; the Brouwer file instantiates it on finite simplex grids and passes to a limit; the product file transports the standard-simplex fixed-point theorem to finite products of simplices; and the Nash file applies that product theorem to mixed strategy spaces.

The formalization also separates reusable infrastructure from proof endpoints. For example, the Scarf layer introduces an indexed family of linear orders before specifying any simplex grid, while the product layer proves its embedding and projection lemmas before the Nash layer uses products of simplices as mixed strategy spaces. This separation is mirrored by BrouwerBench: many benchmark items ask why an intermediate object is needed before the endpoint theorem can be applied.

B. Scarf Proof Components

Table 4 summarizes the combinatorial objects that carry the Scarf proof. The first layer is order theoretic: `isDominant` says that a finite set of points is controlled by a finite set of order indices. The second layer refines dominant pairs into cells, rooms, and doors by cardinality. The third layer equips these pairs with a coloring and counts typed nearly colorful incidences.

The room-door degree facts are the local combinatorial core. Outside doors have one adjacent room, while internal doors have two. After fixing a missing type, the parity proof partitions `dbcountingset` on the door side into outside and internal doors, and on the room side into colorful and non-colorful typed nearly colorful rooms. The outside contribution is odd, while the internal-door and non-colorful-room contributions are even. The remaining colorful-room contribution is therefore odd and hence nonzero.

C. Brouwer Proof Components

The standard-simplex proof converts the finite Scarf theorem into an analytic fixed point. Table 5 records the main proof dependencies.

The final proof endpoint uses two kinds of estimates. On the stabilized color/index set C , colorful witnesses and continuity give coordinate inequalities $(fz)_i \geq z_i$. Outside C , dominance estimates show that the coordinates of z vanish. Since z and fz both lie in the simplex, the sum-one constraints leave no extra mass: the inequalities on C become equalities, the outside coordinates are zero, and extensionality yields $fz = z$.

D. Product-Simplex Proof Components

The product extension reduces a fixed-point problem on a finite product of simplices to a fixed-point problem on one larger simplex. The reduction requires explicit coordinate bookkeeping and a projection whose blockwise denominators are positive.

The positivity argument is the nontrivial part of projection. A raw point in the big simplex may have insufficient mass in a block, so blockwise normalization cannot be treated as ordinary coordinate projection. The push construction moves points toward a uniform interior point. If the push amount is positive, positive block weight contributes directly. If the push amount is zero, zero deficit shows that the original block already meets its positive target weight. These cases justify the denominator in `project_to_product` and allow the continuity proof to use division by a nonzero continuous function.

E. Nash Proof Components

The Nash layer turns the product fixed-point theorem into equilibrium existence. The finite game supplies one simplex per player, and the Nash map raises weights on profitable pure deviations before renormalizing each player block.

At a fixed point of `nash_map`, a profitable pure deviation would make the normalization denominator strictly larger than one. The proof then finds a nonzero coordinate with no positive improvement. Fixedness gives an equality of the form $\sigma_s = \sigma_s / \text{sum_g}$, and `self_div_lemma` forces $\text{sum_g} = 1$, contradicting the strict inequality. The remaining pure-deviation bound extends to arbitrary mixed deviations through `mixed_g_linear`.

F. BrouwerBench Dataset

BrouwerBench v1 is stored as JSONL. Every row corresponds to one hand-checkable question about a local proof role in the formalization. The 80 items are distributed across the same four proof layers as the paper.

Table 3. File-level structure of the formalization.

File	Main objects	Main endpoint	Role in pipeline
Scarf.lean	IndexedLOrder, cells, rooms, doors	Scarf	Finite parity theorem for colorful rooms
Brouwer.lean	TT, Fcolor, room sequences	Brouwer	Grid approximation and limiting fixed point
Brouwer_product.lean	block sums, projection, embedding	Brouwer_Product	Retraction from a big simplex to a product
Nash.lean	mixed games, Nash map, expected payoff	ExistsNashEq	Fixed point implies mixed Nash equilibrium

Table 4. Scarf proof components and their roles.

Lean object	Proof role	Used to establish
IndexedLOrder	family of orders indexed by colors	dominance relation
isDominant	order-theoretic cell condition	finite cell structure
isCell, isRoom, isDoor	cardinality refinements	room–door incidence graph
isDoorof	relation between a door and adjacent room	incidence counting
isOutsideDoor, isInternalDoor	boundary/interior split	degree one versus degree two
isColorful	cell has exactly its color image	target objects
isNearlyColorful, isTypedNC	one fixed missing color	fiberwise parity argument
dbcounitingset	typed door–room incidences	double counting
typed_colorful_room_odd	odd colorful-room count	nonempty colorful filter
Scarf	endpoint theorem	colorful room exists

F.1. Dataset Schema

Each item contains the fields listed in Table 10. The context and question fields are shown to the model together with a section-level Lean prelude. The reference answer and rubric are used for grading rather than included in the prompt.

F.2. Example Items

Scarf definition-role item. Item scarf_001 supplies the local context

isCell abbreviates isDominant; isRoom adds equal point/index cardinalities; isDoor adds one extra index.

It asks how isDominant, isCell, isRoom, and isDoor relate. A score-2 answer must explain dominance as the core cell condition, state the two cardinality refinements, and connect them to the room–door parity structure.

Product endpoint item. Item product_011 provides the lifted map

$$f_{\text{lifted}} = \text{embed_from_product} \circ f \circ \text{project_to_product}$$

and asks why the fixed point is first found in BigSimplex rather than directly in ProductSimplices. A score-2 answer must explain that the available Brouwer theorem is

applied to the lifted single-simplex map, then projection and project_embed_id transport the equation back to the product.

Nash endpoint item. Item nash_011 gives the contradiction branch in ExistsNashEq: the hypothesis $1 < \text{sum_g}$, the fixed-point equation for nash_map, and self_div_lemma. A complete answer explains how a nonzero coordinate gives $\sigma_s = \sigma_s / \text{sum_g}$, why self_div_lemma forces $\text{sum_g} = 1$, and how this contradicts the profitable-deviation branch.

G. Scoring Protocol

The benchmark uses manual item-level scoring. Each item has a rubric tailored to its proof role, but all rubrics follow the shared scale in Table 11.

The grading distinguishes a mathematically plausible answer from a formalization-specific one. An answer that correctly describes a classical theorem but ignores the supplied Lean objects receives partial credit only when it still captures an essential proof dependency. Hallucinated Lean claims, such as applying the wrong endpoint theorem or inventing an unsupported proof route, lower the score. Exact spelling of every Lean identifier is not required when the mathematical dependency is unambiguous, but named objects matter when the question is specifically about their proof role.

The current study uses one manual score file per model out-

Table 5. Key objects in the Scarf-to-Brouwer argument.

Lean object	Local meaning	Global use
<code>TT n 1</code>	finite integer grid with coordinate sum 1	finite domain for Scarf
<code>TTtoStdSimplex</code>	divides grid coordinates by 1	realizes grid points in the simplex
<code>TT.ILO</code>	indexed lexicographic orders on the grid	Scarf order structure
<code>Fcolor</code>	coordinate with $x_i \leq f(x)_i$	grid coloring
<code>room_seq</code>	colorful room at each grid level	discrete approximations
<code>room_point_seq</code>	selected point from each room	sequence to compactify
<code>gpkg</code>	fixed color/index set and subsequence	stable coordinate split
<code>hpkg</code>	convergent subsequence package	limit point z
<code>tendsto_diam_to_zero</code>	rooms shrink along the grid	transfer between room points
<code>dominant_coords_tend_to_zero</code>	coordinates outside fixed set vanish	outside-support control
<code>f_coords_ge_z_coords</code>	inequalities pass to the limit	endpoint equality argument
<code>Brouwer</code>	standard-simplex theorem	fixed point $f \ z = z$

Table 6. Product-simplex constructions.

Lean object	Purpose	Key dependency
<code>index_split, index_combine</code>	convert flat and blockwise indices	inverse lemmas for sums
<code>blockSum, blockWeight</code>	measure mass in each block	positive target block weights
<code>deficit, tPush</code>	measure and bound mass correction	continuous push amount
<code>pushTowardsZ</code>	move toward a uniform interior point	positive post-push block sums
<code>project_to_product</code>	normalize each block	nonzero block denominators
<code>embed_from_product</code>	place product coordinates in big simplex	prescribed block masses
<code>project_embed_id</code>	projection after embedding is identity	fixed-point transport
<code>Brouwer_Product</code>	product fixed-point theorem	lift, apply Brouwer, project back

put. This makes errors auditable at item level: the scored JSONL artifacts store the answer, rubric, score, and score note together. As discussed in §7, a larger benchmark should add independent graders and adjudication for borderline partial-credit cases.

H. Detailed Benchmark Results

The main text reports overall and section-level scores. Table 12 gives the task-type breakdown. It shows that the evaluation probes several kinds of proof understanding rather than a single style of question.

The stems in Table 13 appear after the shared dataset prefix `brouwerbench_v1__`. Raw outputs use the result directory and the `.jsonl` suffix, while manual score files use the score directory and the `.manual.jsonl` suffix. The full per-item responses, manual score notes, scored JSONL files, and generated Markdown reports are retained in the benchmark artifact directory. This keeps the paper tables compact while preserving the item-level audit trail used for the reported totals.

I. Reproducibility Notes

The reported runs were produced locally with Ollama on an Apple M4 Pro machine with 24 GB memory. The Lean ver-

sion is pinned by the repository `lean-toolchain` file and package versions are recorded in `lake-manifest.json`. Benchmark validation checks JSONL parsing, unique ids, rubric fields, source and prelude links, evidence anchors, and whether task contexts leak proof-body material.

The main local workflow is:

```
make -C benchmarks validate
make -C benchmarks run MODEL=qwen3:8b
make -C benchmarks score MODEL=qwen3:8b
```

The GPT run reported in this paper uses the longer output budget:

```
cd benchmarks
python3 scripts/run_ollama_benchmark.py \
  --model gpt-oss:20b \
  --dataset data/brouwerbench_v1.jsonl \
  --output <gpt-np4096-result-path> \
  --num-predict 4096 \
  --overwrite
```

The reported GPT run uses this single budget for all 80 items. Row `product_008` still exhausts the thinking budget, so its empty response is retained and scored as a failure.

After the matching manual score file is present, the scor-

Table 7. Objects in the Nash endpoint.

Lean object	Local meaning	Endpoint role
Game, FinGame	strategic-form data with finiteness	finite mixed space
mixedS	product of player simplices	fixed-point domain
mixed_g	expected payoff	payoff comparison
g_function	current weight plus positive payoff gain	Nash-map numerator
nash_map_aux	normalized coordinate	simplex coordinate
nash_map_cert	sum-one and nonnegativity certificate	mixed profile output
nash_map_cont	continuity of Nash map	Brouwer hypothesis
mixed_g_linear	mixed payoff as pure-deviation average	pure-to-mixed step
self_div_lemma	normalization contradiction	rules out profitable pure deviations
ExistsNashEq	endpoint theorem	mixed Nash equilibrium exists

Table 8. BrouwerBench v1 coverage by formalization section.

Section	Scarf	Brouwer	Product Brouwer	Nash	Total
Items	24	20	18	18	80

ing script merges model responses with rubric scores and writes the scored JSONL and report:

```
python3 scripts/score_benchmark.py \
  --dataset data/brouwerbench_v1.jsonl \
  --results <raw-result-jsonl> \
  --scores <manual-score-jsonl>
```

Section-level prompt preludes are stored as Lean-style snippets rather than standalone Lean modules. They expose the relevant definitions, structures, abbreviations, and inductive declarations to the model while avoiding proof-body leakage. The snippets are therefore prompt context artifacts; they are validated by the benchmark tooling but are not compiled as independent Lean files.

Table 9. BrouwerBench v1 task types.

Type	Def. role	Proof dep.	Parity	Summary	Constr. role	Analysis	Endpoint
Items	7	28	4	3	17	11	10

Table 10. Fields in one BrouwerBench JSONL item.

Field	Meaning
id	stable item identifier
section	one of Scarf, Brouwer, Brouwer_product, or Nash
task_type	proof-role category used for analysis
source	Lean source file from which the item is drawn
section_prelude	Lean-style declaration snippet prepended to the prompt
context	compact task-specific formal context
question	natural-language query answered by the model
gold_answer	reference explanation used during grading
evidence	file-line anchors used to create and check the item
rubric	item-specific 0–2 criteria and required ideas

Table 11. Shared BrouwerBench score scale.

Score	Interpretation	Typical case
0	misses the formal proof role	generic Brouwer or Nash explanation unrelated to the given Lean context
1	partially identifies the role	names a subsequence but misses why the fixed set C is needed
2	explains the Lean-grounded role	connects the named object, dependency, and proof endpoint

Table 12. BrouwerBench v1 scores by task type.

Model	Def.	Dep.	Parity	Summary	Constr.	Analysis	Endpoint
gpt-oss:20b	9/14	45/56	7/8	6/6	26/34	15/22	14/20
qwen3:8b	8/14	38/56	5/8	4/6	23/34	13/22	10/20
gemma3:12b	5/14	34/56	5/8	4/6	20/34	11/22	10/20
kimina-prover:7b	6/14	20/56	1/8	1/6	16/34	9/22	5/20

Table 13. Reported model artifact stems and decoding settings.

Model	Shared artifact stem	Decoding note
gpt-oss:20b	gpt-oss_20b_np4096	num_predict=4096
qwen3:8b	qwen3_8b	default short-answer run
gemma3:12b	gemma3_12b	default short-answer run
kimina-prover:7b	kimina-prover_7b	default short-answer run